

VAST DB 对 LLM Agent 漏洞分析能力的增强效果评估

1 引言

真实程序中的内存漏洞常常具有较长的 source-to-sink 路径和复杂的跨函数数据流。以 use-after-free 为例，释放操作和后续解引用之间可能隔着多层封装函数；悬空指针也可能存放在嵌套的结构体字段、数组或容器对象中，并随这些对象在不同函数间传递。到达 sink 时，局部变量名和代码形态未必还能直接反映它与 source 之间的关系。现有 LLM agent 通常先用 grep 按函数名、变量名或错误位置搜索候选代码，再用 read 阅读局部上下文。这种方式容易检索到名称相近但语义无关的代码片段，也容易在读完局部函数后缺少明确的下一步导航方向，进而遗漏真实路径或转向错误路径。

VAST DB 是一个存储 MLIR 的图数据库，将源码位置、函数调用关系和 def-use 边组织为可查询的图结构，为 LLM 提供一种超越名称匹配的代码导航方式。围绕当前关注的源码位置、符号或 operation，agent 可以沿调用边扩展上下文，判断当前代码由哪些调用路径到达；也可以沿 def-use 边逐步回溯值的来源，找到支撑后续代码阅读的上游定义。数据库查询并不替代源码阅读，而是不断提示下一步值得核对的源码位置，使 agent 能够沿调用关系和数据依赖推进分析。本实验以真实内存 CVE 的漏洞解释任务作为检验场景，对比只使用文本阅读和搜索工具的 baseline agent，以及能够接入 VAST DB 的 VASTDB agent 在 100 个测试用例上的表现。结果显示，VASTDB agent 将准确率从 62.6% 提升至 86.6%。

2 实验设计

2.1 测试用例

实验测试用例来自真实 CVE，覆盖多种内存安全问题：UAF (CWD-1025、CWD-1026)、NPD (CWD-1030、CWD-1031、CWD-1039)、BOF (CWD-1016、CWD-1028、CWD-1029) 和 Memory Leak (CWD-1027)。

测试用例选择偏向 source-to-sink 路径较长、alias 关系较复杂的 CVE。此类漏洞的触发路径往往跨越多个函数，相关对象可能隐藏在结构体字段、数组元素或容器成员中，也可能通过指针别名和回调路径继续传播。agent 如果只阅读 sink 附近的局部代码，通常难以判断控制流和数据流如何从 source 走向 sink。

为了在保留真实分析难度的同时控制实验规模，每个 CVE 被整理为一个自包含的编译单元。构造过程围绕漏洞触发路径抽取相关函数定义，并加入与触发路径相邻或形态相似的干扰函数定义。为保证抽取后的代码能够独立分析和编译，相关函数依赖的类型定义、宏定义、全局变量定义和函数声明也会一并打包。由此得到的测试用例降低了完整工程的规模，同时保留真实 CVE 的核心触发路径和部分噪声。

2.2 VASTDB agent

VASTDB agent 通过 skill 和 MCP tool 两部分接入 VAST DB。skill 负责把数据库结构、查询模板和查询策略暴露给 LLM，使其能够根据当前代码分析目标写出 Cypher 查询语句；MCP tool 则只负责执行 LLM 写出的查询语句，并返回数据库结果。

主要的查询生成能力由 write-cypher skill 提供。该 skill 内置了一组可复用的 Cypher 查询模板，并要求 LLM 在有合适模板时优先复用模板中的查询形状，在没有合适模板时再基于 schema 自行构造查询。模板覆盖了 VAST DB 中最常用的代码导航模式，包括：(1) Location to Operations，用于把源码行或宏展开位置映射为数据库中的 operation，通常作为查询的起点；(2)

Name to Operations, 用于根据函数名、变量名或类型名定位对应的定义 operation, 同样通常作为查询的起点; (3) **Call Chain**, 用于查询函数之间的直接调用关系, 包括从函数定义查找调用者, 以及从一次调用 operation 找到被调函数定义; (4) **Dataflow: Use-Def Chain**, 用于从一次 use 回溯其 value 的来源, 包括局部变量、参数、调用返回值、全局符号引用等; (5) **Dataflow: Def-Use Chain**, 用于从一个 definition 出发查找后续被使用或修改的位置, 包括被传入调用参数、被赋值、赋值给其他变量等; (6) **Code Structure**, 用于从 operation 找到所在函数, 或从函数中找出参数声明、return 等特定子 operation。查询结果通常包含三类有用信息: **location** 指出后续源码阅读中值得关注的位置, **uid** 可作为后续数据库查询的索引, **name** 帮助 LLM 判断下一步执行哪种查询。

write-cypher 还提供了一个组合使用查询模板的导航状态机, 用于约束 LLM 不要只做一次孤立查询。典型流程从 LLM 当前关注的源码位置开始, 例如初始状态下漏洞元信息给出的 source 和 sink。LLM 先用 **Location to Operations** 找到该行对应的多个 operation, 再选择与当前代码元素匹配的 operation uid, 例如分析解引用时选择 **hl.deref**。拿到该 uid 后, LLM 可以进入 **Dataflow: Use-Def Chain** 查询被解引用 value 的定义; 如果返回的是 **hl.param**, 则继续用 **Enclosing Function** 找到参数所属函数, 再用 **Direct Caller** 查找向该参数传值的调用点; 如果返回的是 **hl.call**, 则用 **Call Operation to Callee** 找到返回值来自哪个被调函数; 如果返回的是 **hl.var**, 则可以继续用 **Dataflow: Def-Use Chain** 查找该变量后续参与赋值或传参的位置。通过这种状态转移, 可以让 LLM 的注意力不断覆盖结构上相关的位置, 而不是停留在初始局部上下文中。

辅助 **skill understand-mlir-schema** 用于帮助 LLM 理解 VAST DB 的图结构和查询返回结果。它描述了数据库中的节点标签、边类型和常用属性, 当 LLM 需要解释查询结果, 或者需要在模板之外构造更具体的 Cypher 查询时, 可以加载该 skill 查明 schema 细节, 避免臆造不存在的节点、关系或属性。

这种接入方式的核心是在发挥 LLM 创造性和保证查询质量之间取得平衡。一方面, 让 LLM 自己写查询语句, 可以在模板不完全适配具体漏洞上下文时, 生成更符合当前分析需求的查询; 另一方面, 受模型能力和对 VAST DB 理解程度的限制, LLM 生成的查询语句质量并不稳定, 因此需要提供可直接复用的模板, 使大多数查询能够遵循已有查询形状, 即使需要新写查询, 也可以参考模板中的写法。这样既保留了 LLM 在代码分析中的灵活性, 也降低了生成无效查询语句的风险。

2.3 实验配置

在输入方面, baseline agent 和 VASTDB agent 使用相同的工作目录, 其中仅包括包含漏洞的编译单元。二者的 prompt 都提供相同的漏洞元信息, 包括 CWE 类型、source 位置和 sink 位置, 用于限定分析任务和定位初始代码区域。元信息不包括 CVE-ID 和漏洞所在项目名, 以避免 LLM 利用训练数据中可能存在的先验记忆直接给出答案。

在能力方面, 两个 agent 使用相同的模型 MiniMax-M2.5, 并都被配置为只能使用本地文件系统访问工具, 包括 read、edit、glob 和 grep。实验禁止访问外部目录, 禁止使用 webfetch、websearch 等网络搜索工具, 也禁止使用 LSP。这样的配置使 baseline agent 只能基于本地编译单元进行文本搜索和局部代码阅读。

VASTDB agent 在上述配置基础上额外允许加载接入 VAST DB 所需的 skill, 并可以使用执行数据库查询语句的 MCP tool。除这一差异外, baseline agent 和 VASTDB agent 的输入、工作目录、普通文件访问能力和输出要求保持一致。因此, 两组结果的差异主要反映 VAST DB skill 和 MCP tool 提供的结构化程序信息对 agent 代码理解过程的影响。

2.4 评估方法

实验使用独立 judge agent 对 baseline 和 VASTDB agent 的输出结果进行评估, 以提高实验执行和统计的效率。judge agent 使用更强的大模型 DeepSeek V4, 以提升对漏洞的理解能力。同时, judge agent 除了能访问漏洞版本代码, 还能访问修复版本代码, 并可以通过 diff 比较二者差异, 辅助判断真实漏洞根因。此外, judge agent 获得的元信息还包括 CVE-ID 和 patch 链接, 且允许使用网络

搜索工具，从而获取外部漏洞描述和补丁语义。

judge agent 对 baseline 和 VASTDB agent 的分析结果分别给出正确性判断、0 到 10 分的质量评分和对应的评分理由。评分重点包括根因定位是否准确、触发路径是否完整、函数栈帧和行号是否可靠、以及是否存在缺乏证据的推测等。实验过程中对 judge agent 的评判结果进行了人工抽样检查，结果表明，在更强模型和更多信息的辅助下，judge agent 通常能够给出较准确的评估意见。

3 实验结果与分析

3.1 实验结果

考虑到 LLM 输出具有随机性，实验对 100 个测试用例重复执行 5 次，共得到 500 次有效评估。表 1 给出了 baseline agent 和 VASTDB agent 在五次实验中的准确率和平均评分。汇总结果显示，baseline agent 的准确率为 62.6%，平均评分为 6.65；VASTDB agent 的准确率为 86.6%，平均评分为 8.01。两者准确率相差 24.0 个百分点，平均评分相差 1.36 分，说明接入 VAST DB 后，agent 在真实漏洞根因判断和解释质量上都有明显提升。从逐次实验看，VASTDB agent 在五次实验中均取得更高的准确率和平均评分。

表 1 五次实验的准确率和平均评分

Run	baseline agent		VASTDB agent	
	Accuracy	Score	Accuracy	Score
All	62.6%	6.65	86.6%	8.01
1	59.0%	6.41	86.0%	7.78
2	62.0%	6.54	87.0%	7.86
3	57.0%	6.34	80.0%	7.60
4	71.0%	7.04	97.0%	8.60
5	64.0%	6.92	83.0%	8.19

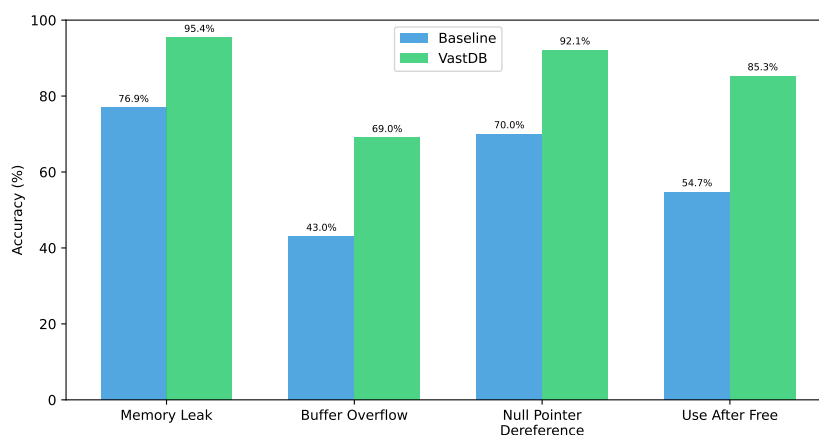


图 1 不同漏洞类型下 baseline agent 与 VASTDB agent 的准确率

图 1 按漏洞类型比较了两种 agent 的准确率。VASTDB agent 在四类漏洞上均取得更高准确率，说明 VAST DB 的收益并不集中在单一漏洞类型上，而是在 Memory Leak、Null Pointer Dereference、Use After Free 和 Buffer Overflow 场景中都能提供帮助。不同类别之间的准确率差异也基本符合任务难度预期：Memory Leak 相对更容易定位，Null Pointer Dereference 和 Use After Free 需要处理更复杂的传播关系，Buffer Overflow 通常涉及更细的边界条件和索引计算，因此整体难度更高。

图 2 进一步展示了按漏洞类型的评分分布。箱体中间的横线表示中位评分，箱体范围表示中间 50% 样例的得分区间，上下须表示非离群样例的分布范围，离群点则对应明显偏低或偏高的个别结果。

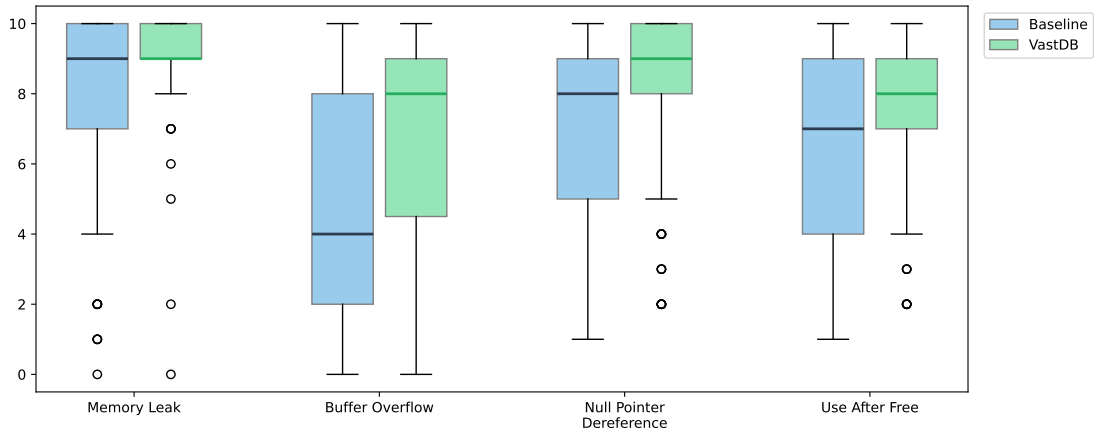


图 2 不同漏洞类型下 baseline agent 与 VASTDB agent 的评分分布

相比 baseline agent, VASTDB agent 的箱体和中位线普遍上移, 说明提升并不是由少数高分样例拉动, 而是多数样例的评分整体变好。同时, VASTDB agent 的箱体整体更窄, 主体分布更靠近高分区间, 低分失败也更少, 表明接入 VAST DB 后评分更集中, 分析质量更稳定。在 Buffer Overflow 这类较难任务上, VASTDB agent 的评分提升更明显; 在 Memory Leak 这类相对简单的任务上, 虽然 baseline agent 本身已经较高, 但 VASTDB agent 仍然能让分布更集中, 减少偶发低分。

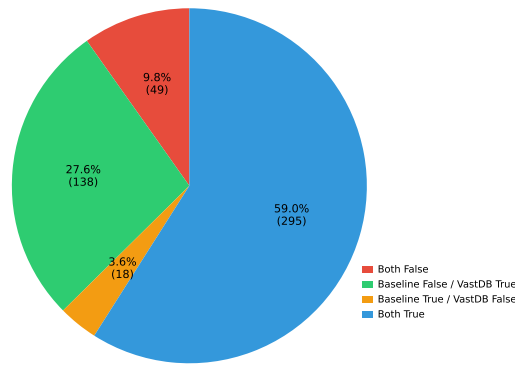


图 3 baseline agent 与 VASTDB agent 的正确性分类结果

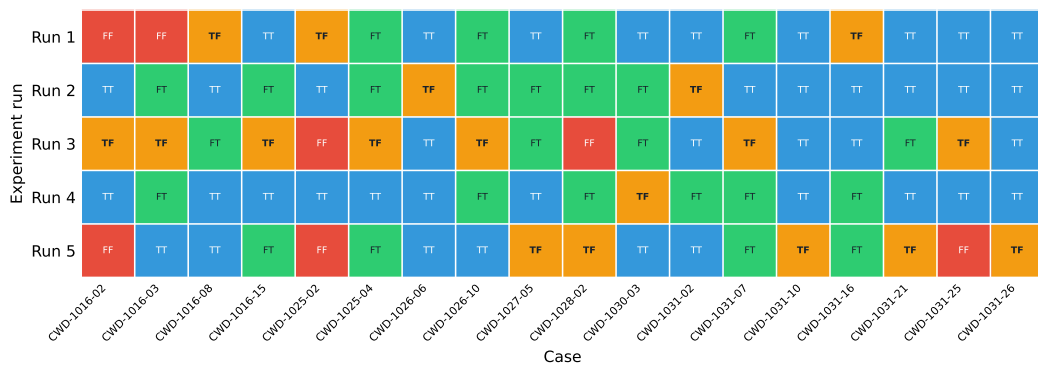


图 4 出现 TF 或 FF 的测试用例在五次实验中的状态变化

图 3 将每次评估结果按 baseline agent 和 VASTDB agent 的正确性分为四类。两者都正确的样例占 59.0%, 说明在多数评估中, 仅依靠文本搜索和局部源码阅读已经能够得到正确分析。baseline agent 错误而 VASTDB agent 正确的样例占 27.6%, 是除两者都正确之外最大的类别, 表明 VAST

DB 的主要收益来自修正 baseline agent 的错误分析；第 3.2 节将进一步按场景展开这些收益来源。baseline agent 正确而 VASTDB agent 错误的样例占 3.6%，比例较低，说明额外接入 VAST DB 整体上没有带来明显的负向影响。

图 4 进一步展示了出现 TF 或 FF 的测试用例在五次实验中的状态变化。可以看到，TF 并不是稳定出现在固定测试用例上的状态，许多 TF 样例在其他重复实验中会转为 TT 或 FT。这说明额外接入 VAST DB 带来的负向影响具有较强偶发性，主要反映 LLM 对 VAST DB 的理解和使用能力仍不稳定：同一测试用例中，agent 有时没有写出正确的数据库查询语句，有时则未能正确解读和利用查询结果。FF 也需要谨慎解释。两者都错误的样例占 9.8%，说明仍存在一部分困难评估；但从热力图看，部分 FF 样例在其他重复实验中会转为 FT 或 TT，因此 FF 不应被简单理解为两种 agent 都稳定无法解决的场景，其中也包含重复实验中的随机性因素。

3.2 VASTDB 的优势场景

前述实验结果表明，VASTDB agent 在整体准确率和评分上均优于 baseline agent。为了分析这种提升的来源，本节选取 baseline agent 分析错误而 VASTDB agent 分析正确的代表性场景，观察 VAST DB 如何改变 LLM 的代码阅读顺序，以及这些阅读顺序变化如何进一步影响根因判断。

这些案例中，baseline agent 的失败通常不是因为完全找不到相关代码，而是因为文本搜索给出的候选位置缺少程序关系上的优先级，LLM 容易停留在表面 source/sink、显眼变量名或局部控制流片段上。VASTDB agent 则将当前关注的位置或符号放入 VAST DB 中继续导航：Location to Operations 和 Name to Operations 用于建立查询起点，Call Chain、Dataflow: Use-Def Chain 和 Dataflow: Def-Use Chain 用于查找调用关系和值传播关系，Code Structure 用于查找操作的所在函数或函数中的某类操作。数据库查询本身不替代源码判断，但会持续给出下一步值得核对的位置，使 LLM 更容易把注意力从局部文本转向结构上相关的代码。表 2 概括了后续各类场景中的代表案例。

表 2 VAST DB 优势场景的代表案例

场景	用例	真实根因或观察点	Baseline 误判	VAST DB 定位
场景 1	CWD-1026-13	读取已失效的 chain 与重新分配底层 chain 两条路径在 bit_copy_chain 汇合	误看局部调用点	定位到汇合函数
	CWD-1031-01	无效 CID 返回 -1，负值写入剩余长度变量后又被当作索引使用	只停在 sink	还原跨函数触发链
场景 2	CWD-1016-13	第二轮编码转换可能返回负数，未检查返回值就累加到输出长度	数据方向判断反了	串起长度计算到写入路径
	CWD-1039-01	union 类型造成的类型混淆产生无效但非 NULL 的指针，绕过 NULL 检查后被使用	简化成 NULL 传参	保留无效非 NULL 指针链
场景 3	CWD-1026-04	UAF 发生在正常所有权转移路径，而不是提前退出的销毁路径	选错候选路径	返回释放链上的关键位置
	CWD-1028-01	运行时 substream 编号范围超过固定数组大小，非独立帧会绕过验证	误判检查覆盖范围	指出检查所在条件块
场景 4	CWD-1026-05	detach/unmap 返回悬空指针后，调用方继续使用该指针	误读互斥分支	带回完整函数上下文
	CWD-1026-14	tuple item 浅拷贝后释放共享数据，留下悬空引用	合并互斥分支结果	定位 tuple item 共享数据

3.2.1 场景 1：表面 source/sink 位置容易误导分析，需要找到真实汇合函数

这类用例中，source 和 sink 只是漏洞路径上的两个端点。真正的根因取决于哪段调用结构将两端连接起来，而不是 source 或 sink 所在的某一行代码。Baseline 容易围绕 source 或 sink 的局部代码收束分析；遇到宏展开、包装函数或多层辅助调用时，又会把表面调用点误认为根因。VAST DB 通过 Location to Operations、Enclosing Function、Direct Caller 和 Call Operation to Callee 查询，将分析单位从单个可疑位置扩展到调用图上的汇合结构。

CWD-1026-13 中，真实汇合点是 bit_copy_chain：一边读取已失效的 chain，另一边通过重新分配修改底层 chain。Baseline 被附近的宏调用点和局部边界细节干扰，没有识别出两条路径在同一上层函数中汇合。VAST DB 查询到读取操作和分配操作都汇入 bit_copy_chain 后，分析焦点收敛

到共享的拷贝/分配路径。CWD-1031-01 展示了更长的跨函数依赖：帧边界解析函数遇到无效 CID 后返回 -1，这个负值被写入剩余长度变量，后续又被当作索引使用，最终因访问空指针而崩溃，调用链跨越 3 个函数。Baseline 识别了 sink，但没有追溯到上游根因。VAST DB 通过 22 次 Cypher 查询还原了完整触发链。

3.2.2 场景 2：关键值的来源或去向超出局部代码窗口

这类用例的根因往往在于同一个值在多次赋值或传递后，其语义已经发生变化。比如，负返回值污染长度变量，指针所有权转移后又被释放，NULL 检查只能过滤 NULL，却无法过滤无效但非 NULL 的指针。Baseline 只看到值的一个端点时，会用变量名、局部表达式或经验模式补全缺失链条，因此容易误判数据方向，也容易误判这个值是在被读取、被写入，还是被传给其他函数。VAST DB 的 Use to Def、One-step Use to Def、Def to Call Argument、Def to Assigned、Def to Assign 和 One-step Def to Use 查询，能把局部端点扩展成一条可逐点核对的值传播路径。

CWD-1016-13 是典型例子。真实问题是第二轮输出循环中的编码转换可能返回负数，代码未检查该返回值就把它累加到输出长度，最终导致越界写入。Baseline 将方向判断反了，认为剩余长度过大导致溢出。VAST DB 将两轮长度计算、分配点、第二轮写入调用和最终的索引写入操作串联到同一条路径上，使两轮循环的差异暴露出来。CWD-1039-01 中，VAST DB 帮助识别 union 类型造成的类型混淆如何产生无效但非 NULL 的指针，并绕过 NULL 检查，而不是把问题简化成 NULL 被直接传入哈希函数。

3.2.3 场景 3：候选路径和执行上下文容易误判

许多漏洞 sink 附近存在多条看似合理的触发路径。Baseline 往往选择文本上最显眼的一条，并将其当作完整根因；有时虽然看到了安全检查、释放或所有权转移等操作，却将其归入错误的条件块或错误处理路径。VAST DB 能准确查找候选调用点、清理/释放位置等关键位置，并可通过 Enclosing Function 确认它们所在的函数，使 LLM 能逐条路径验证和排除。

CWD-1026-04 中，Baseline 选择了提前退出的销毁路径，但真实 UAF 发生在正常所有权转移路径：上层写入函数创建 image 后交给脚本处理流程；中间的 push 操作将原始指针存入状态对象；销毁流程释放该 image 后，上层函数返回时又释放了同一对象。VAST DB 同时返回脚本处理调用点、push 操作、状态销毁和最终释放等关键位置，支持逐段核对并排除错误候选路径。CWD-1028-01 则是执行上下文误判：运行时 substream 编号范围超过固定数组大小。Baseline 找到了边界检查，却遗漏该检查只在独立帧条件块内执行，非独立帧会绕过验证。VAST DB 指出检查的条件性，以及运行时计数器与固定数组大小之间的差异。

3.2.4 场景 4：控制流分支被误读为顺序执行

当漏洞相关代码位于 if-else 链或多路分支中时，Baseline 按连续文本片段阅读源码，容易照着文本顺序理解执行流，将互斥分支中的操作解释为前后相继的事件。VAST DB 并不直接替代控制流推理，但它通过 Location to Operations 返回的精确 operation 位置和 Enclosing Function 将 LLM 引回完整函数上下文，使互斥分支结构更容易被核对。

CWD-1026-05 中，相关代码位于互斥分支：失败分支释放 blob，成功分支才会调整 blob 大小，两者不可能同时执行。Baseline 多次认为释放后会继续执行到 resize 分支，构造出释放后使用的路径；Judge 明确指出该分析描述了不存在的控制流。VAST DB 的定位结果指向 detach/unmap 返回悬空指针后，调用方继续使用该指针。CWD-1026-14 中也出现同类问题：Baseline 将互斥分支中的两次 tuple 求值解释成同一返回值被先后处理，而 VAST DB 将分析引向 tuple item 的浅拷贝问题：共享数据被释放后留下悬空引用。

这些案例里，VAST DB 的价值是把检索入口从松散文本匹配提前到结构化的调用关系和数据依赖上。LLM 仍需回到源码验证最终结论，但 VAST DB 能更早给出可核对的高相关位置，减少上下文浪费，降低路径误判。

4 总结

本文以真实内存 CVE 的漏洞解释任务为场景，对比了仅使用文本搜索和源码阅读工具的 baseline agent，以及额外接入 VAST DB 的 VASTDB agent。实验在相同输入、相同模型和相同普通文件访问能力下进行，使两组结果的差异主要来自 VAST DB 查询能力。五次重复实验显示，VASTDB agent 的准确率从 baseline agent 的 62.6% 提升到 86.6%，平均评分也从 6.65 提升到 8.01，说明 VAST DB 能够稳定改善 LLM agent 的漏洞根因分析质量。

案例分析进一步表明，VAST DB 在源码阅读过程中提供程序关系驱动的导航。它能从当前关注的位置或符号出发，沿调用关系、def-use 关系和代码结构返回下一批值得核对的位置，使 LLM 更容易跳出局部文本匹配的限制。在 source/sink 位置具有误导性、关键值跨函数传播、候选路径和执行上下文容易混淆、互斥分支容易被误读等场景中，这种导航能力能够减少无关上下文消耗，引导 LLM 更快定位真实根因。